

PLUGIN — Security Audit Checklist

Target system: PLUGIN EV charging platform — `plugin-backend` (Spring Boot 3.2 + MongoDB), `plugin-frontend` (React/Vite web), `plugin_app` (Expo / React Native).

Purpose: a hand-off document. Give this to an AI agent or a human reviewer and have them prove, item by item, that the system is not broken — then fix what is.

Version: 1.0 · Generated for repository `antiplugin`, branch `main`.

0. How to use this document

0.1 Instructions for the reviewing agent

You are auditing a live EV-charging platform that holds real people's identity documents, vehicle data, location pings, wallet balances and payment records. Treat every item below as unproven until you have read the code and produced evidence.

Follow these rules exactly:

- **Verify, do not assume.** For every item, open the referenced files and read the actual control flow. Do not mark an item PASS because a comment, a README, or `docs/SECURITY-HARDENING.md` claims it is handled.
- **Prefer an executable proof.** Where a test can express the item, write one under `plugin-backend/src/test/java/com/plugin/` following the existing `*SecurityTest` naming, or extend `plugin-frontend/scripts/security-check.mjs` / `plugin_app/scripts/security-check.cjs`. A passing test is stronger evidence than an argument.
- **Never weaken an existing control to make an item pass.** If a check in `security-check.mjs` or `security-check.cjs` starts failing after your change, the change is wrong — not the check.
- **Report one of four states per item:** `PASS` (with file:line evidence), `FAIL` (with a concrete exploit path), `N/A` (with the reason), `NEEDS-EVIDENCE` (control looks present but cannot be proven without infrastructure — name the infrastructure).
- **Fix in priority order:** CRITICAL, then HIGH, then MEDIUM, then LOW. Do not batch a hundred cosmetic changes ahead of one authorization hole.
- **Keep the suites green.** `mvn clean verify` in `plugin-backend`, `npm test` in `plugin-frontend`, and `npm test` in `plugin_app` must all pass after every change.
- **Do not change public API shapes** (routes, request/response field names) unless an item explicitly requires it — three clients depend on them.
- **Scope discipline.** Fix the security defect. Do not refactor, rename or restyle adjacent code in the same change.

0.2 Severity meaning

- **CRITICAL** — direct account takeover, cross-account data exposure, money loss, or free charging. Blocks release.
- **HIGH** — privilege escalation under a realistic precondition, PII leak, or integrity loss in billing/booking state.
- **MEDIUM** — abuse amplification, weak defence-in-depth, information disclosure that assists another attack.
- **LOW** — hardening, hygiene, or operational gap with no direct exploit today.

0.3 Path shorthand used throughout

- **BE** = `plugin-backend/src/main/java/com/plugin/`
 - **BE-TEST** = `plugin-backend/src/test/java/com/plugin/`
 - **BE-CFG** = `plugin-backend/src/main/resources/application.yml`
 - **FE** = `plugin-frontend/src/`
 - **APP** = `plugin_app/src/`
-

1. Assets, actors and trust boundaries

Establish these before testing anything; every authorization item is judged against them.

Roles (`BE/enums/Role.java`): `CUSTOMER`, `STATION_OPERATOR`, `ADMIN`. There is **no role hierarchy** — `ADMIN` does not implicitly grant `STATION_OPERATOR` and vice versa. Any code that assumes otherwise is a finding.

Assets, ranked by damage if exposed or altered:

- Identity and business KYC files (`StationManagerApplicationFile`, `StationManagerApplicationDocument`) — highest sensitivity, real identity documents.
- Wallet balance, ledger, Razorpay order/payment identifiers (`Wallet`, `WalletLedgerEntry`, `WalletTopUpAttempt`).
- Customer PII — name, email, phone, vehicle registration (`User`, `UserVehicle`).
- Live location pings tied to a booking (`BookingLocationPingRequest`, `BookingLocationPrivacy`).
- Charging session state and meter-derived billing (`ChargingSession`, `Bill`, `Pricing`).
- Admin capability — station CRUD, pricing, customer status toggle, KYC approval, credential issuance, audit logs.

Trust boundaries:

- The browser and the mobile app are **fully untrusted**. Every value they send is attacker-controlled — including IDs, amounts, timestamps, kWh readings, status strings and file metadata.
 - The physical charger is currently a **stub** (`BE/service/ChargerCommandService.java`). Until authenticated OCPP exists, nothing described as "reported by the charger" is trustworthy — see section 17.
 - Razorpay is a third party reachable over the network; its client-side success payloads and callbacks are untrusted until server-verified.
 - The reverse proxy / load balancer is trusted **only** for the CIDRs listed in `TRUSTED_PROXY_CIDRS`.
-

2. P0 sweep — run these eight first

If any of these fail, stop and fix before working through the rest. These are the highest-damage failure modes, expressed against this specific codebase.

☐ P0-01 Cross-account read or write of any owned record

CRITICAL

- Where: `BE/service/` — `BookingService`, `SessionService`, `BillService`, `WalletService`, `NotificationService`, `StationManagerApplicationService`.
- Attack: log in as customer A with a valid token, then call every ID-taking endpoint with an ID belonging to customer B — `GET /api/bookings/{id}`, `GET /api/sessions/{id}`, `GET /api/bills/{id}`, `GET /api/bills/my/{id}/invoice`, `POST /api/bookings/{id}/cancel`, `PUT /api/bookings/{id}`, `POST /api/sessions/end/{sessionId}`, `PATCH /api/notifications/{id}/read`, `POST /api/bills/my/{id}/wallet-pay`.
- Pass: every call returns 403 or 404, leaks no field of B's record, and changes no state. Ownership is resolved from the JWT subject, never from a request parameter or body field.

☐ P0-02 Role escalation through the route matrix

CRITICAL

- Where: `BE/config/SecurityConfig.java`, `BE-TEST/config/SecurityConfigRouteMatrixTest.java`.
- Attack: with a `CUSTOMER` token, call every `/api/admin/**` route. With a `STATION_OPERATOR` token, call `/api/admin/customers`, `/api/admin/revenue`, `/api/admin/audit-logs`, `/api/admin/station-manager-applications/**`, `/api/admin/bills/**`, `/api/admin/sessions/**`, `/api/admin/bookings/**`.
- Pass: 403 in every case. `STATION_OPERATOR` reaches only `/api/admin/dashboard`, `/api/admin/stations/**`, `/api/admin/charging-points/**`, `/api/admin/pricing/**`.

☐ P0-03 NoSQL injection through request-supplied query values

CRITICAL

- Where: every `*RepositoryImpl.java` under `BE/repository/`, plus every `@RequestParam` / `@PathVariable` that reaches a `Criteria` or `Query`.
- Attack: send operator-shaped payloads where a string is expected — `{"email": {"$ne": null}}`, `{"email": {"$gt": ""}}`, `{"password": {"$regex": "^"}}` — to `POST /api/auth/login`, `POST /api/auth/forgot-password`, and every search parameter (`GET /api/stations/search`, `GET /api/admin/customers?search=`, `GET /api/admin/bookings`). Also send a JSON object or array where a `String` is declared.
- Pass: the request is rejected at binding/validation (400), or the value is bound as a literal `String` and never interpreted as a query operator. No endpoint builds a query from a raw JSON fragment or a concatenated string.

☐ P0-04 Login endpoint hardening

CRITICAL

- Where: `BE/service/AuthService.java`, `BE/config/SecurityRateLimitFilter.java`, `BE/dto/request/LoginRequest.java`.
- Attack: 100 rapid failed logins for one account and from one IP; login with a deactivated account; login with an unknown email; login with a 10 MB password field; login with `email` sent as an array or object.
- Pass: bcrypt verification runs on every path (no early return that skips hashing for an unknown user); 429 with `Retry-After` once the window is exceeded; identical response for unknown-email and wrong-password; deactivated accounts refused; DTO size and shape constraints enforced.

☐ P0-05 Token forgery and token lifetime

CRITICAL

- Where: `BE/config/JwtService.java`, `BE/config/JwtAuthFilter.java`.
- Attack: submit a token with `alg: none`; a token signed with an empty or guessed key; a valid token with `role` edited to `ADMIN`; a token belonging to a deleted or deactivated user; a token issued before a password change; a token with a foreign `iss` or `aud`.
- Pass: all rejected with 401. Role and identity are re-read from the current `User` document rather than trusted from the claim, and `tokenVersion` invalidates tokens issued before a password, role or status change.

□ P0-06 Payment and wallet credit integrity

CRITICAL

- Where: `BE/service/RazorpayPaymentService.java`, `BE/service/WalletService.java`, `BE/controller/WalletController.java`.
- Attack: call `POST /api/wallet/topup/verify` with a forged signature; with a valid signature but an inflated amount; replay a previously used `razorpay_payment_id`; submit a negative or zero top-up; race two concurrent verifications of the same order.
- Pass: HMAC verified server-side; amount, currency and capture status re-fetched from Razorpay and compared; payment IDs uniquely constrained so a replay is a no-op; amounts validated positive and bounded; concurrent verification credits exactly once.

□ P0-07 Free or under-charged energy

CRITICAL

- Where: `BE/service/SessionService.java`, `BE/service/BillService.java`, `BE/service/PricingSnapshotService.java`.
- Attack: start a session for a booking you do not own; end a session with a client-supplied kWh of `0`, a negative value, or an enormous value; end the same session twice; start two sessions on one connector; supply a `pricingId` or a rate in the request body.
- Pass: session start requires an owned, eligible booking; energy and price derive from server-side state and a pricing snapshot, never from the request body; end is idempotent; the connector claim is atomic.

□ P0-08 KYC document exposure

CRITICAL

- Where: `BE/service/StationManagerFileService.java`, `BE/service/StationManagerApplicationService.java`, `BE/controller/AdminStationManagerApplicationController.java`, `BE/controller/StationManagerApplicationController.java`.
 - Attack: fetch `GET /api/station-manager/application/files/{slotType}` and `.../business-documents/{documentType}/file` as a different customer and as an unauthenticated user; guess a `referenceId` at `GET /api/station-manager/status/{referenceId}`; path-traverse the `slotType` / `documentType` segment (`..%2f..%2fetc%2fpasswd`).
 - Pass: files resolve only through the authenticated owner's application or an `ADMIN`; enum-bound slot and document types reject traversal; status lookup requires the authenticated owner and returns minimal fields only.
-

3. Authentication and account lifecycle

Covers `BE/controller/AuthController.java`, `BE/service/AuthService.java`, `ForgotPasswordService.java`, `OtpSecurityService.java`, `BE/config/PasswordPolicy.java`, `IdentityNormalizer.java`, and DTOs under `BE/dto/request/`.

☐ AUTH-01 Registration cannot claim an existing email

HIGH

- Attack: `POST /api/auth/register` twice with the same email; with the same email in different case (`A@b.com` vs `a@b.com`); with Unicode homoglyphs; with leading/trailing whitespace.
- Pass: `IdentityNormalizer` canonicalises before lookup; a unique index on `users.email` backs it; the second attempt neither overwrites nor links the existing account.

☐ AUTH-02 Registration response does not confirm account existence

MEDIUM

- Attack: register with an email already in use and compare status, body and latency against a fresh email.
- Pass: responses are indistinguishable, or registration is intentionally enumeration-tolerant with a documented decision and rate limits to match.

☐ AUTH-03 Role cannot be self-assigned at registration

CRITICAL

- Attack: `POST /api/auth/register` with `{"role": "ADMIN"}`, `{"role": "STATION_OPERATOR"}`, `{"active": true, "tokenVersion": 99}`, `{"id": 1}`.
- Pass: `RegisterRequest` has no role/active/id field and the service hard-codes `Role.CUSTOMER`. Jackson must not bind unknown fields onto the entity — confirm the DTO is the only bound type and `User` is never deserialised from a request body.

☐ AUTH-04 Password policy is enforced server-side

HIGH

- Where: `BE/config/PasswordPolicy.java`.
- Attack: register and change password with `a`, `password`, `12345678`, a 1 MB string, and a string of only spaces.
- Pass: minimum length and complexity enforced on the server for register, reset, change-password and station-manager access setup alike; maximum length bounded (bcrypt truncates at 72 bytes — reject or pre-hash rather than silently truncating).

☐ AUTH-05 Passwords are stored with bcrypt only

CRITICAL

- Attack: read `BE/config/SecurityConfig.java` `passwordEncoder()` and every write path to `User.password`.
- Pass: `BCryptPasswordEncoder` everywhere; no MD5/SHA/plaintext fallback; no path writes a raw password; work factor is explicit and at least 10.

☐ AUTH-06 Pending registrations expire and cannot be resurrected

MEDIUM

- Where: `BE/entity/PendingRegistration.java`, `BE/repository/PendingRegistrationRepository.java`.
- Attack: create a pending registration, wait past expiry, then `POST /api/auth/confirm-otp`; also confirm the same OTP twice.
- Pass: expired records are refused and cleaned up; confirmation is single-use and atomic.

☐ AUTH-07 OTP values are unguessable and peppered

CRITICAL

- Where: `BE/service/OtpSecurityService.java`, `BE/entity/PasswordResetOtp.java`, `BE-CFG app.otp.*`.
- Attack: read the generation code — is it `Random` or `SecureRandom`? Is the raw OTP stored? Is `OTP_PEPPER` required at startup?
- Pass: `SecureRandom`, at least 6 digits, only an HMAC of the OTP stored (peppered with `OTP_PEPPER`), startup fails without the pepper.

□ AUTH-08 OTP attempt limit and cooldown are atomic

HIGH

- Attack: 1000 parallel `POST /api/auth/forgot-password/verify-otp` guesses against one OTP; `POST /api/auth/resend-otp` in a tight loop.
- Pass: attempts increment atomically (`findAndModify` , not read-then-write), the OTP is destroyed at `app.otp.max-attempts` (default 5), and resend honours `app.otp.resend-cooldown-seconds` (default 60) per account and per IP.

□ AUTH-09 OTP is consumed exactly once

HIGH

- Attack: verify one valid OTP twice concurrently; verify then reuse the same OTP for a different action (delete-account OTP used for password change).
- Pass: atomic consume-on-verify; OTPs are bound to a purpose and cannot be swapped between the register / forgot-password / change-password / delete-account flows.

□ AUTH-10 Forgot-password does not reveal account existence

HIGH

- Where: `BE/service/ForgotPasswordService.java` , `BE-TEST/service/ForgotPasswordServiceSecurityTest.java` .
- Attack: request a reset for a known and an unknown email; compare status, body, headers and elapsed time.
- Pass: identical responses; unknown addresses do not short-circuit into a fast path.

□ AUTH-11 Reset token/OTP cannot be redirected to another account

CRITICAL

- Attack: `POST /api/auth/forgot-password/reset` with account A's OTP and account B's email; with a mismatched `referenceId` .
- Pass: the OTP record's own bound identity determines the target account; the request-supplied email is only compared, never used to select the victim.

□ AUTH-12 Password change and reset revoke every existing token

HIGH

- Where: `User.revokeSessions()` , `tokenVersion` .
- Attack: log in on two devices, change the password on one, then use the other device's token.
- Pass: the second token returns 401. The same must hold for reset, role change, deactivation and account deletion.

□ AUTH-13 Change-password requires the current password or a verified OTP

HIGH

- Where: `BE/controller/ProfileController.java` `/change-password*` , `ChangePasswordRequest` , `ChangePasswordOtpRequest` .
- Attack: change the password with only a bearer token and no current password / OTP proof; reuse a stale OTP verification.
- Pass: a fresh proof of possession is required, its verification window is short, and it is single-use.

□ AUTH-14 Google sign-in verifies the ID token properly

CRITICAL

- Where: `BE/service/AuthService.java` Google path, `BE-TEST/service/AuthServiceGoogleSecurityTest.java` , `BE-CFG app.google.*` .
- Attack: post an unsigned token; a token signed by another issuer; a valid Google token minted for a different `aud` ; a token with `email_verified: false` ; an expired token.
- Pass: signature verified against Google's keys, `aud` matched against `GOOGLE_CLIENT_IDS` exactly, issuer checked, expiry enforced, unverified emails refused, timeouts bounded (`GOOGLE_CONNECT_TIMEOUT_MS` , `GOOGLE_READ_TIMEOUT_MS`).

□ AUTH-15 Google sign-in cannot take over a privileged account by email

CRITICAL

- Where: `User.googleSubject` , `BE-TEST/service/AuthServiceIdentitySecurityTest.java` .
- Attack: sign in with Google using the email address of an existing `ADMIN` / `STATION_OPERATOR` password account.
- Pass: linking requires a matching stored `googleSubject` ; an email match alone never grants the existing account, and never auto-creates a privileged role.

□ AUTH-16 Deactivated and deleted accounts cannot authenticate

HIGH

- Attack: deactivate a user via `PATCH /api/admin/customers/{id}/status`, then log in and use a pre-existing token; repeat for a deleted account.
- Pass: both login and token validation check `active` and existence on every request.

□ AUTH-17 Station-manager access invitations are single-use and short-lived

CRITICAL

- Where: `BE/entity/StationManagerAccessInvitation.java`, `BE/service/StationManagerAccessSetupService.java`, `BE-TEST/service/StationManagerAccessSetupServiceTest.java`.
- Attack: use one invitation token twice; use it after expiry; brute-force the token; read the database to see whether the raw token is stored; request `POST /api/station-manager/access/setup` for a token belonging to another application.
- Pass: only a SHA-256 hash is stored, the raw token has ≥ 128 bits of entropy and travels in a URL fragment, consumption is atomic and single-use, expiry is short (hours, not days), and comparison is constant-time.

□ AUTH-18 No temporary or emailed passwords exist anywhere

HIGH

- Attack: grep for `temporaryPassword`, `generatePassword`, `randomPassword` across backend and frontend; read `StationManagerCredentialEmailService.java`.
- Pass: approval issues an invitation link only; no generated password is stored, logged, emailed or rendered.

□ AUTH-19 Account deletion is authenticated, confirmed, and complete

HIGH

- Where: `ProfileController` `/delete`, `/delete/send-otp`, `/delete/verify-otp`, `/delete/forgot*`.
- Attack: delete another user's account by ID; delete with only a bearer token and no confirmation; check what survives deletion (bills, sessions, wallet, KYC files, notifications).
- Pass: only the authenticated subject can be deleted, a fresh OTP or password is required, tokens are revoked, and the retention decision for financial records is deliberate and documented rather than accidental.

□ AUTH-20 The delete-account "forgot" path is not an unauthenticated delete

CRITICAL

- Attack: call `POST /api/profile/delete/forgot/send-otp` and `/delete/forgot` for an arbitrary email without being logged in as that user.
- Pass: the route is authenticated (it sits under `/api/profile/**`), and the OTP is bound to the authenticated subject — not to an email supplied in the body.

□ AUTH-21 Auth DTOs constrain size and shape

MEDIUM

- Where: `BE/dto/request/*Request.java`, `BE-TEST/dto/request/AuthDtoSecurityValidationTest.java`.
- Pass: `@NotBlank`, `@Email`, `@Size` on every auth field; `@Valid` on every controller parameter; oversized bodies rejected before hitting business logic.

□ AUTH-22 No credential material reaches logs

HIGH

- Attack: grep for `log.` statements in `AuthService`, `ForgotPasswordService`, `OtpSecurityService`, `JwtService`, `StationManagerAccessSetupService`; run a failing login and a failing OTP at `DEBUG`.
 - Pass: no password, OTP, raw invitation token, JWT, or Razorpay secret is ever logged, at any level.
-

4. Tokens and session management

Covers `BE/config/JwtService.java`, `JwtAuthFilter.java`, `BE-TEST/config/JwtServiceTest.java`, `JwtAuthFilterTest.java`.

☐ TOK-01 Signing algorithm is pinned

CRITICAL

- Attack: `alg: none`; `alg: HS256` on a service expecting RS256 or vice versa; a JWT with an unexpected `kid`.
- Pass: the parser accepts exactly one algorithm; algorithm is never read from the token header to select the verification path.

☐ TOK-02 Secret strength is enforced at startup

HIGH

- Pass: the application refuses to start when `JWT_SECRET` is absent or shorter than 32 bytes; there is no default, fallback or dev value in code or `BE-CFG`.

☐ TOK-03 `iss` and `aud` are validated

HIGH

- Attack: token with `iss: attacker`, `aud: other-app`, or those claims absent.
- Pass: exact match against `JWT_ISSUER` / `JWT_AUDIENCE`; a missing claim is a rejection, not a skip.

☐ TOK-04 Expiry and clock skew are enforced

HIGH

- Where: `BE/config/AppClock.java`.
- Attack: an expired token; a token with `exp` far in the future; a token with `nbf` in the future.
- Pass: expiry enforced with minimal allowed skew; 24 h access-token life (`app.jwt.expiration-ms`) is reviewed against risk, and a shorter life with refresh is the target state.

☐ TOK-05 Claims are re-bound to the current user record

CRITICAL

- Attack: edit `role`, `email`, `userId` in a validly signed token.
- Pass: `JwtAuthFilter` loads the `User` and derives the authority from the stored role; a mismatch between claim and record is a 401, not a silent preference for one side.

☐ TOK-06 `tokenVersion` revocation works

HIGH

- Attack: token issued at version N, bump the user's version, retry.
- Pass: 401. Confirm every mutation that should revoke (password change/reset, role change, deactivation, deletion, KYC approval that changes role) calls `revokeSessions()`.

☐ TOK-07 Bearer parsing is strict

MEDIUM

- Attack: `Authorization: bearer <t>`, `Bearer <t>` (double space), `Bearer <t> <t2>`, token in a query string `?token=`, token in a cookie.
- Pass: one canonical header form is accepted; tokens are never read from query strings (they leak into access logs and Referer).

☐ TOK-08 Filter failures fail closed

CRITICAL

- Attack: make the user lookup throw (drop Mongo) while presenting a valid token.
- Pass: 401/503, never an authenticated context. No `catch (Exception e) { chain.doFilter(...) }` that proceeds unauthenticated.

☐ TOK-09 Filters are not double-registered

MEDIUM

- Where: the `FilterRegistrationBean` disablers in `SecurityConfig`.

- Pass: `JwtAuthFilter` and `SecurityRateLimitFilter` run only inside the Spring Security chain — verify they do not also execute as plain servlet filters on permitted routes.

□ TOK-10 Rate-limit filter ordering cannot be bypassed

HIGH

- Attack: hammer `/api/auth/login` with a malformed `Authorization` header so JWT parsing fails early.
- Pass: rate limiting still applies. Confirm the ordering (`addFilterAfter(rateLimit, JwtAuthFilter)`) cannot let a JWT failure short-circuit the limiter for unauthenticated endpoints.

□ TOK-11 Sessions are genuinely stateless

LOW

- Pass: `SessionCreationPolicy.STATELESS` holds and nothing sets a `JSESSIONID`; no server-side session state is relied upon.

□ TOK-12 CSRF disablement is justified

HIGH

- Pass: CSRF is disabled only because authentication is a non-cookie bearer header. If any cookie-based auth is ever introduced, CSRF protection must return. Confirm no endpoint accepts credentials from a cookie today.

□ TOK-13 No token appears in a URL, log or error body

HIGH

- Attack: trigger 401/403/500 responses and inspect bodies and server logs; check `GlobalExceptionHandler`.
- Pass: tokens never echoed; error bodies are generic.

□ TOK-14 Web and mobile logout fully clear credentials

MEDIUM

- Where: `FE/utils/authStorage.js`, `FE/context/AuthContext.jsx`, `APP/api/client.js`.
 - Pass: logout clears the token, cached user, offline session snapshot (`plugin_active_session_snapshot`) and any remembered API base URL.
-

5. Authorization, ownership and the route matrix

The definitive route rules live in `BE/config/SecurityConfig.java`. Route-level rules are necessary but **not sufficient** — ownership must also be enforced in services.

□ AZ-01 Reconstruct and test the full route matrix

CRITICAL

- Method: for each route below, issue requests as (a) anonymous, (b) `CUSTOMER`, (c) `STATION_OPERATOR`, (d) `ADMIN`, and for each of GET/POST/PUT/PATCH/DELETE. Record the status code. Extend `BE-TEST/config/SecurityConfigRouteMatrixTest.java` with any gap.
- Expected policy:
 - Public: `POST /api/auth/register|login|google|confirm-otp|resend-otp|forgot-password|forgot-password/send-otp|forgot-password/verify-otp|forgot-password/reset`; `GET /api/stations/**`; `GET /api/charging-points/station/**`; `GET /api/pricing/station/**`; `GET /api/station-manager/reference-data`; `POST /api/station-manager/access/setup`; `OPTIONS /**`.
 - `CUSTOMER` only: `/api/bookings/**`, `/api/sessions/**`, `/api/bills/**`, `/api/wallet/**`.
 - `CUSTOMER` or `STATION_OPERATOR`: `GET /api/station-manager/status/**`, `POST /api/station-manager/application`, other `/api/station-manager/**`.
 - `STATION_OPERATOR` only: `POST /api/station-manager/session`.
 - `CUSTOMER`, `ADMIN` or `STATION_OPERATOR`: `/api/profile/**`.
 - `ADMIN` or `STATION_OPERATOR`: `/api/admin/dashboard`, `/api/admin/stations/**`, `/api/admin/charging-points/**`, `/api/admin/pricing/**`.
 - `ADMIN` only: `/api/admin/station-manager-applications/**` and every other `/api/admin/**` (customers, bookings, sessions, bills, revenue, audit-logs).
 - Authenticated (any role): `/api/notifications/**`.
- Pass: no route is reachable by a role outside its row; anything unlisted falls through to `authenticated()` and nothing is accidentally public.

□ AZ-02 Non-GET methods under public namespaces stay closed

CRITICAL

- Attack: `POST /api/stations`, `PUT /api/stations/1`, `DELETE /api/stations/1`, `POST /api/pricing/station/1`, `PATCH /api/charging-points/station/1`, `GET /api/auth/login`.
- Pass: the explicit `denyAll()` rules cover them — 403, not 404 or 200.

□ AZ-03 Path-matching cannot be tricked

HIGH

- Attack: `/api/admin../admin/customers`, `/api/Admin/customers`, `/api/admin/customers/`, `/api/admin//customers`, `/api/admin/customers%2f`, `/api/admin/customers;jsessionid=x`, a trailing `.json`.
- Pass: all normalise to the protected rule or 400; none reach a controller unauthorised. Confirm Spring's path normalisation and `AntPathRequestMatcher` behaviour under the deployed servlet container.

□ AZ-04 Ownership is enforced in the service layer, not only by route

CRITICAL

- Where: every service method that takes an ID.
- Method: list every public service method with an `id` parameter and confirm each loads the record **and** compares its owner to the authenticated principal before reading or mutating.
- Pass: no method trusts that "the route was CUSTOMER-only, therefore this booking is mine".

□ AZ-05 The principal comes from the security context, never the request

CRITICAL

- Attack: send `userId`, `customerId`, `email` or `ownerId` in bodies and query strings on booking, session, bill, wallet, notification and profile endpoints.
- Pass: those fields are ignored entirely; grep for any controller reading an identity from `@RequestParam /body`.

□ AZ-06 STATION_OPERATOR is scoped to its own stations

CRITICAL

- Where: `BE/service/StationOperatorAccessService.java`, `StationManagerDirectoryService.java`, `AdminController` `station/point/pricing` methods.
- Attack: as operator X, `PUT /api/admin/stations/{id}` on operator Y's station; `POST /api/admin/charging-points` with Y's `stationId`; `DELETE /api/admin/charging-points/{id}` for Y's point; `POST /api/admin/pricing` for Y's station; `GET /api/admin/bookings/station/{stationId}` for Y's station; `GET /api/admin/charging-points/station/{stationId}` for Y's station.
- Pass: 403 in every case. This is the most likely remaining multi-tenant hole — test every operator-reachable route with a foreign station ID.

□ AZ-07 Operators cannot create stations they do not own

HIGH

- Attack: as operator X, `POST /api/admin/stations` with an owner/manager field pointing at Y, or with no owner so it defaults to a global pool.
- Pass: ownership is assigned from the authenticated operator, and a supplied owner field is ignored or rejected.

□ AZ-08 Admin-only capabilities are truly admin-only

CRITICAL

- Attack: as `STATION_OPERATOR`, `PATCH /api/admin/customers/{id}/status`, `POST /api/admin/station-manager-applications/{id}/approve`, `.../issue-credentials`, `GET /api/admin/audit-logs`, `GET /api/admin/revenue`, `POST /api/admin/bills/{id}/pay`, `POST /api/admin/sessions/{id}/end`.
- Pass: 403.

□ AZ-09 No mass-assignment path to privileged fields

CRITICAL

- Attack: add `role`, `active`, `tokenVersion`, `balance`, `status`, `amount`, `paid`, `id`, `mongoId` to every `PUT` / `POST` body — profile update, booking create/reschedule, KYC application, wallet auto-topup, station create.
- Pass: request DTOs expose only user-editable fields; entities are never bound directly from request bodies; unknown fields are ignored or rejected consistently.

□ AZ-10 Numeric legacy IDs are not treated as capabilities

HIGH

- Where: `BE/entity/MongoSequence.java`, `MongoSequenceService.java`.
- Attack: enumerate sequential IDs on every resource — bookings, bills, sessions, stations, applications — as a low-privileged user.
- Pass: sequential IDs are usable only with a passing ownership check; nothing is protected by ID unguessability alone.

□ AZ-11 Method-level security is used where route rules are coarse

MEDIUM

- Where: `@EnableMethodSecurity` is on; check whether `@PreAuthorize` is applied on sensitive service methods.
- Pass: at minimum, admin approval, credential issuance, customer status change and pricing writes carry an explicit method-level guard as defence in depth.

□ AZ-12 Notifications are per-user

HIGH

- Where: `BE/service/NotificationService.java`, `BE-TEST/service/NotificationServiceSecurityTest.java`.
- Attack: `PATCH /api/notifications/{id}/read` for another user's notification; `GET /api/notifications` and check for foreign rows; `PATCH /api/notifications/read-all` and check for cross-user effect.
- Pass: every query and mutation is filtered by the authenticated user ID.

□ AZ-13 Pagination and filters cannot widen scope

HIGH

- Attack: on `/api/bookings/my`, `/api/bills/my`, `/api/sessions/my`, `/api/notifications`, `/api/admin/*` list endpoints — pass `size=100000`, `page=-1`, `sort=password`, `userId=<other>`, `status=` unknown enum, unbounded date ranges.
- Pass: page size is capped, sort fields are allow-listed (never free-form field names that could expose `password`), date ranges are bounded, and the owner filter is always applied server-side.

□ **AZ-14** Error codes do not become an authorization oracle

MEDIUM

- Attack: compare responses for "record does not exist" vs "record exists but belongs to someone else".
 - Pass: both return the same status and body for non-admin callers, so IDs cannot be enumerated by response differences.
-

- Pass: filenames are server-generated or strictly sanitised; header values never contain raw user input.

□ INJ-09 CSV / spreadsheet formula injection

HIGH

- Where: `GET /api/bills/my/statement`, admin exports.
- Attack: set a profile field, vehicle registration or station name to `=cmd|'/c calc'!A1`, `+1+1`, `-1+1`, `@SUM(1)`, then export and open the CSV.
- Pass: cells beginning with `= + - @ TAB CR` are neutralised (prefixed or quoted) on export.

□ INJ-10 Server-side request forgery in outbound calls

HIGH

- Where: Google token verification, Razorpay client, email/SMTP, any URL built from configuration or user data.
- Attack: check whether any user-supplied value can influence an outbound URL or host.
- Pass: outbound hosts are fixed constants or validated allow-lists; timeouts are bounded; redirects are not blindly followed.

□ INJ-11 Email/template injection

MEDIUM

- Where: `InvoiceEmailService.java`, `StationManagerCredentialEmailService.java`, `StationManagerTrackingEmailService.java`.
- Attack: set your full name to `<script>`, to a CRLF sequence, or to a `${...}` expression, then trigger an email.
- Pass: no header CRLF injection, user values are escaped in HTML bodies, and no template engine evaluates user input as an expression.

□ INJ-12 PDF generation cannot be driven by user input

MEDIUM

- Where: `BE/service/InvoicePdfService.java`.
- Attack: long, RTL, control-character and markup-laden values in name/address/station fields on an invoice.
- Pass: values are escaped/truncated, generation cannot be made to load a remote resource, and a malformed value causes a clean error rather than a crash loop.

□ INJ-13 Multipart limits are enforced

HIGH

- Where: `BE-CFG` `spring.servlet.multipart.max-file-size` (10 MB) / `max-request-size` (50 MB), plus service-level checks.
- Attack: upload a 2 GB file; 100 parts in one request; a zip-bomb-style highly compressed image; a part with no filename; duplicate part names.
- Pass: limits hold at the container level and are re-checked in `StationManagerFileService`; exceeding them returns 413 with no partial write.

□ INJ-14 Deserialization surface is minimal

MEDIUM

- Pass: Jackson polymorphic typing (`@JsonTypeInfo`, default typing) is not enabled; no endpoint accepts Java-serialised or YAML input; unknown properties are handled by an explicit, consistent policy.

□ INJ-15 Global exception handler leaks nothing

HIGH

- Where: `BE/exception/GlobalExceptionHandler.java`, `BE-TEST/exception/GlobalExceptionHandlerTest.java`.
- Attack: trigger a Mongo error, a validation error, a `NullPointerException`, and a 500 on every controller family; inspect the bodies.
- Pass: no stack trace, class name, query fragment, collection name, or file path reaches the client; a correlation ID is returned instead and the detail goes to the log.

□ INJ-16 Unicode and normalisation abuse

MEDIUM

- Attack: register with a homoglyph or a right-to-left override in the email/name; use a Turkish dotless `ı` in a case-insensitive comparison; submit lone surrogates.

- Pass: `IdentityNormalizer` applies a consistent normalisation form before storing and comparing, and comparisons use a locale-independent case fold.
-

7. Booking flow

Covers `BE/controller/BookingController.java`, `BE/service/BookingService.java`, `EtaService.java`, `StationOperatingHoursPolicy.java`, `BookingLocationPrivacy.java`, `BE-TEST/service/BookingOverlapTest.java`.

□ BOOK-01 Booking creation binds to the authenticated customer

CRITICAL

- Attack: `POST /api/bookings` with a `userId / customerId` field naming another account.
- Pass: the owner is taken from the security context and the supplied field is ignored.

□ BOOK-02 Double-booking a connector is impossible

HIGH

- Attack: fire N concurrent `POST /api/bookings` for the same connector and overlapping window; repeat against a replica set with transactions enabled.
- Pass: exactly one succeeds. Verify the guard is a database-level atomic claim or a transaction with a unique constraint — not an in-process lock, which fails across instances.

□ BOOK-03 Booking windows are validated server-side

HIGH

- Attack: start in the past; end before start; a 30-day duration; a zero-length booking; a window outside station operating hours; a booking on an inactive station or an out-of-service connector.
- Pass: all rejected by `StationOperatingHoursPolicy` and explicit range checks against `AppClock`.

□ BOOK-04 Booking quantity abuse is limited

MEDIUM

- Attack: create 500 bookings across all connectors of a station to deny service to real customers.
- Pass: a per-customer concurrent/pending booking cap exists, plus a rate limit on creation.

□ BOOK-05 Reschedule requests cannot bypass approval

HIGH

- Where: `POST /api/bookings/{id}/reschedule-request`, `POST /api/admin/bookings/{id}/reschedule/approve|reject`, `RescheduleRequestStatus`.
- Attack: request a reschedule on someone else's booking; approve your own reschedule as a `CUSTOMER`; approve twice; approve a rejected request; reschedule into an occupied slot.
- Pass: ownership on request, `ADMIN`-only approval, one-way state transitions, and overlap re-checked at approval time (not only at request time).

□ BOOK-06 Cancellation is owner-bound and idempotent

HIGH

- Attack: cancel another customer's booking; cancel an already-cancelled or completed booking; cancel after a session has started; cancel to dodge a bill.
- Pass: ownership enforced, state machine rejects invalid transitions, and any refund/credit path is idempotent.

□ BOOK-07 Admin cancellation is audited and admin-only

MEDIUM

- Where: `POST /api/admin/bookings/{id}/cancel`, `AdminBookingCancelRequest`.
- Pass: `ADMIN` only, reason recorded, an `AuditLog` entry written with the acting admin's identity.

□ BOOK-08 Location pings are owner-bound, rate-limited and minimised

HIGH

- Where: `POST /api/bookings/{id}/location`, `BookingLocationPingRequest`, `BookingLocationPrivacy.java`.
- Attack: ping another customer's booking; ping a completed booking; ping 1000 times a second; ping with coordinates out of range or with excessive precision; read back another user's last-known location through any endpoint.
- Pass: ownership enforced, accepted only while the booking is in an active window, rate-limited, coordinates range-validated, precision reduced, retention bounded, and never exposed to another customer.

□ BOOK-09 Location data is not exposed to operators or admins beyond need

HIGH

- Attack: as `STATION_OPERATOR` and `ADMIN`, look for raw customer coordinates in `/api/admin/bookings`, `/api/admin/bookings/station/{id}`, dashboards and exports.
- Pass: coordinates are either absent or coarsened; ETA is derived server-side and only the derived value is shared.

□ BOOK-10 Booking status transitions are a closed state machine

HIGH

- Where: `BE/enums/BookingStatus.java`.
- Attack: `PUT /api/bookings/{id}` with an arbitrary `status`; move `CANCELLED` → `CONFIRMED`; `COMPLETED` → `PENDING`.
- Pass: status is never client-settable; transitions are validated centrally against an explicit allowed-transition map.

□ BOOK-11 Booking expiry job cannot double-fire across instances

HIGH

- Where: `BE-CFG app.bookings.expiry-*`.
- Attack: run two backend instances against one database and watch for duplicated expiry effects (double notifications, double refunds, double state changes).
- Pass: a distributed lease/fencing token guards the scheduler. This is an open item in `docs/SECURITY-HARDENING.md` — confirm status and record it.

□ BOOK-12 Booking reads are owner-filtered including counts

HIGH

- Attack: `GET /api/bookings/my` with injected filters; `GET /api/bookings/{id}` for a foreign booking; check any count/stat endpoint for cross-user leakage.
- Pass: owner filter applied in the query itself, not by post-filtering a broader result set.

□ BOOK-13 Booking responses do not over-share

MEDIUM

- Where: `BE/dto/response/BookingResponse.java`.
- Pass: the DTO exposes only what the client needs — no internal Mongo `_id` where a public ID exists, no other user's identity, no pricing internals that enable manipulation.

□ BOOK-14 Pricing is snapshotted at booking time and not re-negotiable

HIGH

- Where: `BE/service/PricingSnapshotService.java`.
 - Attack: create a booking, have an operator lower the price, then complete — and vice versa; supply a `pricingId` or `ratePerKwh` in the booking body.
 - Pass: the snapshot is server-captured and immutable; the client cannot influence which pricing applies.
-

8. Charging sessions

Covers `BE/controller/SessionController.java`, `BE/service/SessionService.java`, `ChargerCommandService.java`, `BE-TEST/service/SessionServiceSecurityTest.java`.

☐ SESS-01 Session start requires an owned, eligible booking

CRITICAL

- Attack: `POST /api/sessions/start/{bookingId}` with another customer's booking ID; with a cancelled, completed or future booking; with a booking whose station is inactive.
- Pass: ownership plus state and time-window eligibility checked; otherwise 403/409.

☐ SESS-02 One active session per booking and per connector

CRITICAL

- Attack: call start twice concurrently for the same booking; start on a connector that already has an active session.
- Pass: an atomic connector claim (conditional update) ensures exactly one winner; the loser gets a clean conflict.

☐ SESS-03 Session end is owner-bound and idempotent

CRITICAL

- Attack: `POST /api/sessions/end/{sessionId}` for a foreign session; end the same session twice concurrently; end an already-completed session.
- Pass: ownership enforced, second end is a no-op, exactly one bill is produced.

☐ SESS-04 Energy and duration are never client-supplied

CRITICAL

- Attack: include `energyConsumed`, `kwh`, `endTime`, `durationMinutes`, `cost` in the end request body.
- Pass: the fields do not exist on the request DTO, and the server computes values from its own clock and state. Any client-influenced energy value is a direct money bug.

☐ SESS-05 Charger command stub is not treated as trusted acknowledgement

HIGH

- Where: `BE/service/ChargerCommandService.java`.
- Pass: it is clearly marked as a stub, cannot silently mark energy delivered, and the production gap (authenticated OCPP, signed meter values, start/stop acknowledgement, disconnect handling) is tracked. Do not claim production readiness while this is a stub.

☐ SESS-06 Session auto-complete job is safe

HIGH

- Where: `BE-CFG app.sessions.auto-complete-*` (runs every 1000 ms).
- Attack: two instances against one database; a session that fails mid-completion.
- Pass: distributed lease, idempotent completion, no duplicate billing, and a tight loop interval that does not itself become a load problem.

☐ SESS-07 `GET /api/sessions/{id}` and `/my` are owner-filtered

CRITICAL

- Attack: request a foreign session ID; enumerate sequential session IDs.
- Pass: 403/404 with no data leak.

☐ SESS-08 Admin session termination is admin-only and audited

MEDIUM

- Where: `POST /api/admin/sessions/{id}/end`.
- Pass: `ADMIN` only (not operator), audit entry written, and the resulting bill follows the same server-side computation path.

☐ SESS-09 Active-session lookup does not leak other users

HIGH

- Where: `GET /api/sessions/my/active`, consumed by `APP/screens/ChargingScreen.js`.
- Pass: strictly scoped to the caller.

□ **SESS-10** Offline session cache cannot be used to fake state

MEDIUM

- Where: `APP` AsyncStorage key `plugin_active_session_snapshot`.
- Attack: modify the cached snapshot on a rooted device to show a session that is not real, or to extend one.
- Pass: the cache is display-only; no server decision ever derives from it; it is cleared on logout; it contains no token and no other user's data.

□ **SESS-11** Session status transitions are closed

HIGH

- Where: `BE/enums/SessionStatus.java`.
- Pass: transitions validated centrally; no client-settable status; terminal states are final.

□ **SESS-12** Concurrency guards use database primitives

HIGH

- Method: grep for `synchronized`, `ReentrantLock`, `AtomicBoolean` in services and confirm none of them is the sole protection for a cross-instance invariant.
 - Pass: correctness relies on optimistic versions (`@Version`), conditional updates, or transactions — the app is expected to run more than one instance.
-

9. Wallet and payments

Covers `BE/controller/WalletController.java`, `BE/service/WalletService.java`, `RazorpayPaymentService.java`, entities `Wallet`, `WalletLedgerEntry`, `WalletTopUpAttempt`, and `BE-TEST/service/WalletServiceSecurityTest.java`, `RazorpayPaymentServiceSecurityTest.java`. This is where money is lost — audit it hardest.

☐ PAY-01 Razorpay signature verification is server-side and constant-time

CRITICAL

- Attack: `POST /api/wallet/topup/verify` with an empty signature, a truncated signature, a signature from a different order, and a signature computed with the public key ID.
- Pass: HMAC-SHA256 over the documented payload using `RAZORPAY_KEY_SECRET`, compared with a constant-time equality function, never `String.equals` on secrets and never skippable.

☐ PAY-02 The order is re-fetched from Razorpay and fully compared

CRITICAL

- Attack: a valid signature for a real ₹1 order, replayed with a ₹100000 amount in the body.
- Pass: the server re-reads the order/payment from Razorpay and compares amount, currency, status (captured), and order-to-user binding. The client-reported amount is never used to credit.

☐ PAY-03 Payment identifiers are unique and replay-proof

CRITICAL

- Attack: submit the same `razorpay_payment_id` / `razorpay_order_id` twice, concurrently and sequentially.
- Pass: a unique index on the payment identifier makes the second attempt a no-op with exactly one ledger entry. Verify the index actually exists in the target database, not just as an annotation.

☐ PAY-04 Orders are bound to the creating user

CRITICAL

- Attack: user A creates an order via `POST /api/wallet/topup/order`, user B verifies it with A's identifiers.
- Pass: verification checks that the stored attempt belongs to the authenticated user; otherwise 403 and no credit.

☐ PAY-05 Amounts are validated at order creation

HIGH

- Attack: `POST /api/wallet/topup/order` with `0`, negative, `0.001`, `1e12`, a string, and below `app.razorpay.minimum-amount`.
- Pass: `BigDecimal` with a minimum, a maximum, and two-decimal scale; rejected values never create an order.

☐ PAY-06 Wallet balance updates are atomic and versioned

CRITICAL

- Attack: concurrent credit and debit on one wallet (top-up verify + bill payment + session debit at once).
- Pass: optimistic locking or conditional atomic updates; the final balance equals the ledger sum under concurrency. Write a concurrency test if one does not exist.

☐ PAY-07 Balance can never go negative

CRITICAL

- Attack: pay a bill larger than the balance; two concurrent debits each within balance but exceeding it together.
- Pass: the debit is a conditional update guarded on sufficient balance, not a read-check-write.

☐ PAY-08 Ledger is append-only and reconciles

HIGH

- Where: `WalletLedgerEntry`, `WalletLedgerType`.
- Attack: look for any update or delete path on ledger entries.
- Pass: entries are immutable, every balance change has exactly one entry, and `sum(ledger) == balance` for every wallet (write a reconciliation check).

☐ PAY-09 Withdrawal cannot be abused

CRITICAL

- Where: `POST /api/wallet/withdraw`, `WalletWithdrawalRequest`.

- Attack: withdraw more than the balance; withdraw a negative amount to credit yourself; two concurrent withdrawals; withdraw to a destination account supplied in the request; withdraw non-refundable promotional credit.
- Pass: amount validated positive and $\leq$ available balance atomically; destination is bound to verified account details, not free-form client input; every withdrawal is audited and, ideally, requires re-authentication.

□ PAY-10 Auto top-up mandates cannot be forged

CRITICAL

- Where: `PUT /api/wallet/auto-topup`, `POST /api/wallet/auto-topup/disable`, `/mandate/order`, `/mandate/verify`, `WalletAutoTopUpRequest`, `WalletMandateOrderRequest`.
- Attack: enable auto top-up on another user's wallet; verify a mandate with another user's identifiers; set a threshold/amount that causes a charge loop; disable another user's mandate.
- Pass: every mandate operation is owner-bound and signature-verified; thresholds and amounts are bounded; the customer/mandate state is confirmed with Razorpay.

□ PAY-11 The test-confirm path is impossible in production

CRITICAL

- Where: `POST /api/wallet/mandate/test-confirm`, gated on `app.production`; `APP/screens/WalletScreen.js` `__DEV__` guard; mobile check line 44 in `security-check.cjs`.
- Attack: call the endpoint against a production-configured instance; check whether the route exists in the deployed build at all.
- Pass: 404/403 when `APP_PRODUCTION=true`; ideally the route is not registered at all in production. Also confirm production startup rejects Razorpay test keys.

□ PAY-12 Webhooks are authenticated and idempotent

CRITICAL

- Status: `docs/SECURITY-HARDENING.md` lists this as an open production blocker. Confirm current state.
- Requirements: a webhook endpoint that verifies Razorpay's webhook signature, tolerates duplicate and out-of-order delivery, uses provider idempotency keys, and reconciles against local state. Relying on the client to report success is not acceptable for a launch.

□ PAY-13 The wallet monitor job is safe

HIGH

- Where: `BE-CFG` `app.wallet.monitor-*` (every 3000 ms).
- Attack: two instances triggering the same auto top-up.
- Pass: distributed lease; a duplicate trigger cannot double-charge the customer.

□ PAY-14 Ledger and wallet reads are owner-scoped and paginated

HIGH

- Attack: `GET /api/wallet`, `GET /api/wallet/ledger` with a foreign user filter, `size=1000000`, `page=-1`, an unbounded date range.
- Pass: owner filter always applied, page size capped, dates bounded.

□ PAY-15 Currency is fixed server-side

MEDIUM

- Attack: send `currency: "USD"` on order creation to exploit a conversion mismatch.
- Pass: currency comes from `app.razorpay.currency`; a client-supplied currency is ignored or rejected.

□ PAY-16 Razorpay keys never reach a client

CRITICAL

- Attack: grep the frontend and mobile bundles for `rzp_`, `key_secret`, `RAZORPAY_KEY_SECRET`; inspect `RazorpayOrderResponse`.
- Pass: only the public key ID is ever returned; the secret exists only in backend configuration.

□ PAY-17 Rounding and precision cannot be farmed

HIGH

- Attack: many micro-transactions to exploit rounding in either direction; a bill of `0.005`.

- Pass: one documented rounding mode applied consistently, `BigDecimal` throughout, no `float` / `double` in any money path (grep for `double` in wallet, bill and pricing code).

☐ **PAY-18 Refund and reversal paths are controlled**

HIGH

- Method: find every code path that credits a wallet without a Razorpay payment (cancellation refunds, admin adjustments, promotional credit).
- Pass: each is admin-authorized or system-generated with a strict trigger, idempotent, audited, and impossible to trigger repeatedly by cancelling and rebooking.

☐ **PAY-19 Payment-related endpoints are rate-limited**

MEDIUM

- Attack: loop order creation to exhaust Razorpay quota or create thousands of pending attempts.
- Pass: per-account rate limits on order creation and verification; stale attempts expire.

☐ **PAY-20 Money-affecting actions are audited**

HIGH

- Pass: top-up, debit, withdrawal, refund, mandate change and admin bill payment each write an `AuditLog` entry with actor, target, amount, and correlation to the provider identifier.
-

10. Billing and invoices

Covers `BE/controller/BillController.java`, `BE/service/BillService.java`, `InvoicePdfService.java`, `InvoiceEmailService.java`, `BE-TEST/service/BillServiceSecurityTest.java`.

☐ BILL-01 Every bill read is ownership-checked

CRITICAL

- Attack: `GET /api/bills/{id}` and `GET /api/bills/my/{id}/invoice` with a foreign or enumerated ID.
- Pass: 403/404, no PDF, no metadata.

☐ BILL-02 Bill amounts are server-computed only

CRITICAL

- Attack: any request field that could influence the amount — `amount`, `rate`, `kwh`, `discount`, `taxRate`.
- Pass: computed from the session and the pricing snapshot; `app.billing.default-rate-per-kwh` and `efficiency-factor` are server configuration, never client input.

☐ BILL-03 Wallet payment of a bill is owner-bound and idempotent

CRITICAL

- Attack: `POST /api/bills/my/{id}/wallet-pay` on a foreign bill; pay the same bill twice concurrently; pay an already-paid bill.
- Pass: ownership enforced, conditional atomic transition to paid, exactly one debit.

☐ BILL-04 Admin bill payment is admin-only and audited

HIGH

- Where: `POST /api/admin/bills/{id}/pay`.
- Pass: `ADMIN` only; audit entry with actor and reason; cannot silently zero a customer's debt without a trace.

☐ BILL-05 Statement export is bounded and owner-scoped

HIGH

- Where: `GET /api/bills/my/statement`, `X-Statement-Count` response header.
- Attack: a 10-year range; `size` beyond the cap; a foreign user filter.
- Pass: date range and row count capped, owner filter enforced, and the exposed count header reveals nothing beyond the caller's own data.

☐ BILL-06 Exported cells are formula-neutralised

HIGH

- See INJ-09. Verify with a live export containing a `=` prefixed field.

☐ BILL-07 Invoice PDFs contain only the owner's data

HIGH

- Attack: generate an invoice and check for any other customer's name, email, phone or vehicle; check whether internal IDs or admin notes leak.
- Pass: scoped strictly to the bill's owner and the necessary station details.

☐ BILL-08 Invoice downloads are attachment-only with safe headers

MEDIUM

- Pass: `Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`, `Cache-Control: no-store`, a server-generated filename, and the correct `application/pdf` content type.

☐ BILL-09 Invoice email goes only to the verified account address

HIGH

- Where: `InvoiceEmailService.java`.
- Attack: influence the recipient through a profile update or a request field; trigger a mail flood by repeated requests.
- Pass: recipient read from the stored, verified user record; sending is rate-limited.

☐ BILL-10 Unpaid-count and stats endpoints are owner-scoped

MEDIUM

- Where: `GET /api/bills/my/unpaid-count`.

- Pass: no cross-user aggregate is reachable by a customer.

☐ **BILL-11 Bills cannot be deleted or edited by a customer**

HIGH

- Method: confirm no route or service path allows a customer to modify bill state other than paying it.
 - Pass: no such path exists.
-

11. Stations, charging points and pricing

Covers `BE/controller/StationController.java`, `AdminController.java`, `BE/service/StationService.java`, `ChargingPointService.java`, `PricingService.java`, and their `*SecurityTest` classes.

□ STN-01 Public station data is minimal

HIGH

- Attack: as an anonymous user call `GET /api/stations`, `/api/stations/search`, `/api/stations/{id}`, `/api/stations/{id}/charging-points`, `/api/stations/{id}/pricing`, `/api/stations/live-summary` and look for operator names, emails, phone numbers, internal IDs, revenue figures, occupancy tied to identifiable customers, or KYC-derived fields.
- Pass: only publicly necessary station attributes are returned. Operator PII in a public response is a real leak — this is the largest anonymous attack surface in the system.

□ STN-02 Live summary does not expose customer identity

HIGH

- Attack: `GET /api/stations/live-summary` while a session is running; look for user IDs, names or booking IDs.
- Pass: aggregate counts only.

□ STN-03 Public endpoints are rate-limited and paginated

MEDIUM

- Attack: scrape the full station catalogue at high rate; request `size=100000`.
- Pass: pagination cap and an anonymous rate limit; scraping is bounded.

□ STN-04 Station writes are role- and owner-checked

CRITICAL

- See AZ-06. Test `POST/PUT/PATCH/DELETE /api/admin/stations/**` and `/api/admin/charging-points/**` as an operator against a foreign station.

□ STN-05 Station deletion is safe

HIGH

- Attack: delete a station with active bookings or a live session.
- Pass: refused or soft-deleted with dependency handling; no orphaned sessions or bills, and no cascade that destroys financial records.

□ STN-06 Charging-point status changes are validated

HIGH

- Where: `PATCH /api/admin/charging-points/{id}/status`, `PointStatus`.
- Attack: set an arbitrary status string; free an occupied connector to break an active session; set status on a foreign station's point.
- Pass: enum-bound, transition-validated, ownership-checked, audited.

□ STN-07 Pricing writes are controlled and versioned

CRITICAL

- Where: `POST /api/admin/pricing`, `DELETE /api/admin/pricing/{id}`, `PricingRequest`, `PricingModel`.
- Attack: as an operator, set a negative or zero rate; set a rate on a foreign station; delete pricing mid-session; set an absurd rate to overcharge.
- Pass: bounded positive values, ownership enforced, changes audited, and in-flight sessions/bookings continue on their snapshot (see BOOK-14).

□ STN-08 Geo queries are bounded

MEDIUM

- Attack: `GET /api/stations/search` with a radius of `1e9`, invalid coordinates, or a NaN.
- Pass: radius capped, coordinates range-validated, result count capped.

□ **STN-09** Station images and static assets are safe

MEDIUM

- Method: check whether stations carry client-supplied image URLs or uploads.
- Pass: URLs are validated (scheme/host allow-list) or uploads follow the KYC file rules in section 12; no `javascript:` or `data:` URL can reach the web client.

□ **STN-10** Operating-hours policy cannot be bypassed

MEDIUM

- Where: `StationOperatingHoursPolicy.java`, `BE-TEST/service/StationOperatingHoursPolicyTest.java`.
 - Attack: book across a midnight boundary, on a closed day, or in a different timezone offset.
 - Pass: server-side evaluation in a canonical timezone.
-

12. Station-manager KYC onboarding and file handling

Covers `BE/controller/StationManagerApplicationController.java`, `AdminStationManagerApplicationController.java`, `StationManagerAccessSetupController.java`, `BE/service/StationManagerApplicationService.java`, `StationManagerFileService.java`, `StationManagerDirectoryService.java`, `StationManagerAccessSetupService.java`, and `BE-TEST/service/StationManagerApplicationSecurityTest.java`, `StationManagerFileServiceSecurityTest.java`.

This section holds the most sensitive data in the product.

☐ KYC-01 Application submission is bound to the authenticated account

CRITICAL

- Attack: `POST /api/station-manager/application` with another user's email/ `referenceId`; submit twice to overwrite an existing application; submit on behalf of another account.
- Pass: the applicant is the authenticated subject; a supplied email is ignored; an existing application cannot be overwritten by another user.

☐ KYC-02 Status lookup requires the owner

CRITICAL

- Attack: `GET /api/station-manager/status/{referenceId}` while unauthenticated; as a different customer; by enumerating reference IDs.
- Pass: authenticated owner only, minimal fields returned (status and timestamps — no PII, no reviewer notes, no documents).

☐ KYC-03 Reference IDs are unguessable

HIGH

- Attack: check whether reference IDs are sequential or derived from a counter; generate two and compare.
- Pass: high-entropy random values; sequential legacy lookup is removed. Even so, unguessability is never the only control (see KYC-02).

☐ KYC-04 File downloads resolve through ownership, never a path

CRITICAL

- Where: `GET /api/station-manager/application/files/{slotType}, .../business-documents/{documentType}/file`.
- Attack: request another applicant's file; traverse with `..%2f`; supply an absolute path; supply an unknown enum value; request while unauthenticated.
- Pass: the file is located from the authenticated owner's application record plus an enum slot; no request value ever forms part of a filesystem path.

☐ KYC-05 Admin file access is `ADMIN`-only and audited

CRITICAL

- Where: `GET /api/admin/station-manager-applications/{id}/files/{slotType}` and `.../business-documents/{documentType}/file`.
- Attack: as `STATION_OPERATOR` or `CUSTOMER`; as an admin, access an application under review by another admin.
- Pass: `ADMIN` only; every identity-document view writes an `AuditLog` entry naming the admin, the application and the document.

☐ KYC-06 Upload type checking uses content, not the declared name

CRITICAL

- Attack: upload `payload.php.jpg`; a real JPEG with an appended PHP/JSP payload; an SVG containing `<script>`; an HTML file renamed `.pdf`; a PDF with JavaScript, `/OpenAction`, or an embedded file; a polyglot GIF/JS file; a 0-byte file; a file with a null byte in the name.
- Pass: magic-byte/signature validation against an allow-list of types, declared MIME and extension both ignored as authority, SVG rejected outright, active PDF constructs rejected.

☐ KYC-07 Images are decoded and re-encoded

HIGH

- Pass: uploaded images are parsed and rewritten server-side so that appended payloads and metadata (including EXIF GPS, which is a privacy leak for an ID photo) do not survive. Verify decode failures are rejected rather than passed through.

□ KYC-08 Stored filenames are server-generated

CRITICAL

- Attack: upload with the name `../../../../etc/passwd`, `../../../../web.config`, a 4000-character name, and a name containing CR/LF.
- Pass: the stored name is a generated identifier; the original name is stored only as escaped metadata, and never used to build a path or a header.

□ KYC-09 Per-file and per-request size limits are enforced

HIGH

- See INJ-13. Also confirm a cap on the number of files per application and total storage per applicant.

□ KYC-10 Files are served with defensive headers

HIGH

- Pass: `Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`, a restrictive sandbox CSP, `Cache-Control: no-store`, and a generic or verified content type — never the client-declared one.

□ KYC-11 No inline preview of KYC files in the web admin

CRITICAL

- Where: `FE/pages/admin/StationManagerApplications/StationManagerApplications.jsx`; enforced by `security-check.mjs` lines 33–35.
- Attack: look for `<iframe>`, `<object>`, `<embed>`, `window.open`, or a blob URL rendered in the page.
- Pass: download-only. A rendered attacker-supplied document is stored XSS against an admin session.

□ KYC-12 Files live outside the web root and outside the repository

CRITICAL

- Method: find the storage directory used by `StationManagerFileService` and confirm it is not served statically by Spring, nginx or CloudFront, and is not inside the deployed artifact.
- Pass: no direct URL can reach the raw file; all access goes through the authorised controller.

□ KYC-13 Malware scanning gap is acknowledged

HIGH

- Status: open blocker in `docs/SECURITY-HARDENING.md` — private encrypted object store, quarantine/clean buckets, AV plus CDR scanning, signed short-lived downloads, retention jobs.
- Pass: either implemented, or explicitly recorded as a launch blocker with an owner. Signature checks are not an antivirus verdict.

□ KYC-14 Approval flow cannot be self-served

CRITICAL

- Where: `POST /api/admin/station-manager-applications/{id}/approve|reject|issue-credentials`.
- Attack: as the applicant, approve your own application; as an operator, approve any application; approve an already-approved application; approve then re-issue credentials repeatedly.
- Pass: `ADMIN` only, one-way state transitions, idempotent, audited, and (target state) requiring recent re-authentication or MFA.

□ KYC-15 Role elevation on approval is deliberate and revokes tokens

CRITICAL

- Attack: after approval, check whether the applicant's role changed to `STATION_OPERATOR` and whether their old `CUSTOMER` token still works.
- Pass: role change is explicit, audited, and calls `revokeSessions()` so the old token cannot be used with a stale role claim.

□ KYC-16 Credential issuance sends an invitation, not a password

CRITICAL

- See AUTH-17 and AUTH-18. Also confirm the invitation email cannot be redirected to an attacker-supplied address.

□ KYC-17 `POST /api/station-manager/access/setup` is safe as a public route

CRITICAL

- Attack: brute-force tokens; submit a weak password; submit for an expired or already-used invitation; time the response to distinguish a valid token prefix.

- Pass: strict rate limiting on this public route, constant-time token comparison, password policy enforced, single-use consumption, and enumeration-resistant responses.

□ **KYC-18** Application list and detail are admin-scoped and paginated

HIGH

- Where: `GET /api/admin/station-manager-applications`, `GET .../{id}`.
- Attack: as a non-admin; with `size=100000`; with a `sort` naming a sensitive field.
- Pass: `ADMIN` only, page capped, sort allow-listed.

□ **KYC-19** Reviewer notes and internal fields never reach the applicant

HIGH

- Where: `StationManagerReviewRequest`, `StationManagerApplicationResponse` vs `StationManagerStatusLookupResponse`.
- Pass: the applicant-facing DTO excludes reviewer identity, internal notes and other applicants' data.

□ **KYC-20** Reference data is safe to expose publicly

MEDIUM

- Where: `GET /api/station-manager/reference-data` (public).
- Pass: it returns only static enums and business rules — no counts of applications, no internal thresholds that assist fraud, no PII.

□ **KYC-21** The operator directory sync runner is safe

HIGH

- Where: `BE/config/StationManagerDirectorySyncRunner.java`.
- Attack: consider what happens on a duplicate or conflicting record at startup, and whether it can grant `STATION_OPERATOR` to an unintended account.
- Pass: idempotent, fails closed on ambiguity, logs without PII, and cannot silently elevate a role.

□ **KYC-22** KYC retention and deletion exist

HIGH

- Method: check what happens to identity documents after rejection, after approval, and after account deletion.
 - Pass: a defined retention window with an actual deletion job, plus legal-hold handling. Indefinite retention of rejected applicants' ID documents is a privacy finding.
-

13. Admin console, analytics and audit trail

Covers `BE/controller/AdminController.java`, `BE/service/AdminCustomerService.java`, `DashboardService.java`, `AuditService.java`, `BE/entity/AuditLog.java`, and the pages under `FE/pages/admin/`.

☐ ADM-01 Every admin route is role-gated at both layers

CRITICAL

- Pass: `SecurityConfig` route rule **and** a service/method-level check. A route rule alone breaks the moment a path pattern is edited.

☐ ADM-02 Customer list exposes the minimum PII

HIGH

- Where: `GET /api/admin/customers`, `AdminCustomerResponse`.
- Attack: check for password hashes, `tokenVersion`, Google subject, full vehicle registration, raw location, and whether an operator can reach it.
- Pass: no credential material ever leaves the server; the response is `ADMIN`-only and field-minimised.

☐ ADM-03 Customer search is injection-safe and bounded

HIGH

- See INJ-03. Test `search=` with regex metacharacters, operator documents and very long strings.

☐ ADM-04 Customer status toggle is protected

CRITICAL

- Where: `PATCH /api/admin/customers/{id}/status`.
- Attack: as an operator; toggle another admin's account; toggle your own account; deactivate every account in a loop.
- Pass: `ADMIN` only, admin accounts protected from casual deactivation, self-deactivation guarded, audited, rate-limited.

☐ ADM-05 Revenue and analytics cannot be scoped-escaped

HIGH

- Where: `GET /api/admin/revenue`, `GET /api/admin/dashboard`, `GET /api/admin/bookings/stats`.
- Attack: as an operator, request `/api/admin/dashboard` and check whether the figures are global or restricted to the operator's own stations; pass a `stationId` for a foreign station.
- Pass: operators see only their own stations' figures; the shared dashboard route is the highest-risk leak in the admin area — verify it explicitly.

☐ ADM-06 Audit log is admin-only, append-only and complete

HIGH

- Where: `GET /api/admin/audit-logs`, `AuditService`, `AuditLog`.
- Attack: as an operator; look for any update/delete path on audit entries; check whether an admin can erase their own actions.
- Pass: `ADMIN` only, immutable entries, and coverage of at least: login failures and lockouts, role changes, status toggles, KYC approve/reject, credential issuance, KYC document views, pricing changes, station/point mutations, admin bill payment, admin session termination, admin booking cancellation, wallet adjustments.

☐ ADM-07 Audit entries identify the actor reliably

HIGH

- Pass: actor identity comes from the security context, includes a request correlation ID and the resolved client IP (via `TrustedClientIpResolver`), and is written in the same transaction as the action where possible.

☐ ADM-08 Audit entries contain no secrets or raw PII

MEDIUM

- Pass: no passwords, OTPs, tokens or full document contents; identity documents are referenced by ID, not embedded.

☐ ADM-09 Admin listing endpoints are paginated and sort-restricted

MEDIUM

- See AZ-13. Applies to customers, bookings, sessions, bills, audit logs and applications.

□ ADM-10 High-impact admin actions require step-up authentication

HIGH

- Status: open item in `docs/SECURITY-HARDENING.md` (MFA and recent re-authentication).
- Target: approval, credential issuance, role/status change, refunds and withdrawals require MFA or a recent re-auth. Record current state honestly.

□ ADM-11 The dead `EmailInbox` view stays out of the build

MEDIUM

- Where: `FE/pages/EmailInbox/EmailInbox.jsx`, `EmailInboxMessage` response DTO; enforced by `security-check.mjs` line 19.
- Attack: check whether any backend route still serves mailbox contents.
- Pass: no route, no import, no reachable page. If a backend endpoint still exists, remove it — a readable inbox is an account-takeover primitive via OTP interception.

□ ADM-12 Admin UI cannot be reached by role-guard bypass

HIGH

- Where: `FE/components/ProtectedRoute/ProtectedRoute.jsx`.
 - Attack: set a forged user object in storage; navigate directly to `/admin/*`.
 - Pass: the client guard is cosmetic — every admin API call is refused server-side regardless of what the browser believes.
-

14. Notifications, profile and vehicles

☐ NOTF-01 Notification content carries no other user's data

HIGH

- Where: `BE/service/NotificationService.java`, `NotificationResponse`.
- Pass: messages reference only the recipient's own bookings, sessions and bills.

☐ NOTF-02 Read/read-all are scoped to the caller

HIGH

- See AZ-12.

☐ NOTF-03 Notification text is not rendered as HTML

HIGH

- Where: `FE/components/NotificationBell/NotificationBell.jsx`, `APP/screens/NotificationsScreen.js`.
- Attack: get `<img src=x onerror=alert(1)>` into a station name or a profile name that feeds a notification.
- Pass: rendered as text; no `dangerouslySetInnerHTML` anywhere in the notification path.

☐ NOTF-04 Notification volume is bounded

MEDIUM

- Attack: trigger notification-generating actions in a loop to flood a victim or the mail service.
- Pass: dedupe and rate-limit; the booking-start scheduler (`app.notifications.*`) cannot re-notify the same booking repeatedly.

☐ NOTF-05 Push/system notifications leak nothing on a lock screen

MEDIUM

- Where: `APP/utils/systemNotifications.js`.
- Pass: no amounts, no addresses, no OTPs in notification bodies.

☐ PROF-01 Profile update cannot change privileged fields

CRITICAL

- Where: `PUT /api/profile`, `ProfileUpdateRequest`.
- Attack: include `role`, `active`, `email`, `id`, `tokenVersion`, `password`, `googleSubject`.
- Pass: the DTO carries only editable fields; email changes (if allowed at all) require re-verification and token revocation.

☐ PROF-02 Profile read returns only the caller

HIGH

- Attack: `GET /api/profile` with an injected `userId`.
- Pass: strictly the authenticated subject.

☐ PROF-03 Vehicles are owner-bound

HIGH

- Where: `DELETE /api/profile/vehicles/{vehicleId}`, `ProfileVehicleRequest`, `UserVehicle`.
- Attack: delete or read another user's vehicle by ID; add 10 000 vehicles.
- Pass: ownership enforced, a per-user count cap, registration format validated and length-bounded.

☐ PROF-04 Vehicle registration is treated as PII

MEDIUM

- Pass: not exposed in public station or session responses, not written to logs, and included in export/deletion flows.

☐ PROF-05 Profile fields are length- and charset-bounded

MEDIUM

- Attack: a 1 MB full name; control characters; RTL overrides; HTML.
- Pass: `@Size` limits and a sane character policy on name and phone.

☐ PROF-06 Phone numbers are validated and not used as an auth factor

MEDIUM

- Pass: format-validated; no flow trusts an unverified phone for recovery.

15. Rate limiting and abuse control

Covers `BE/config/SecurityRateLimitFilter.java`, `BE/service/SecurityRateLimitService.java`, `TrustedClientIpResolver.java`, `BE/entity/SecurityRateLimitBucket.java`, `BE-CFG app.rate-limit.*`, and their tests.

☐ RATE-01 The limiter fails closed

HIGH

- Attack: make the rate-limit collection unavailable and replay login attempts.
- Pass: 503, never an unlimited pass-through. Confirm `RATE_LIMIT_ENABLED=true` in production and that disabling it is impossible without a config change.

☐ RATE-02 Client IP resolution cannot be spoofed

CRITICAL

- Where: `TrustedClientIpResolver.java`, `TRUSTED_PROXY_CIDRS`, `server.forward-headers-strategy: none`.
- Attack: send `X-Forwarded-For: 1.2.3.4`, a long XFF chain, `X-Real-IP`, `Forwarded:` from an untrusted source; rotate spoofed values to defeat per-IP limits.
- Pass: forwarded headers are ignored unless the socket peer is in `TRUSTED_PROXY_CIDRS`; the trusted list contains only the immediate proxy; the edge overwrites inbound forwarded headers.

☐ RATE-03 Bucket keys are peppered and non-reversible

MEDIUM

- Pass: keys are HMACs using `RATE_LIMIT_PEPPER`; startup fails without it; raw emails and IPs are not stored as plaintext bucket identifiers.

☐ RATE-04 Counter increments are atomic

HIGH

- Attack: 200 concurrent requests at the limit boundary.
- Pass: atomic `$inc` with an upsert and a TTL — never read-then-write.

☐ RATE-05 Both per-IP and per-account limits exist on credential endpoints

HIGH

- Endpoints to verify: `/api/auth/login`, `/register`, `/google`, `/confirm-otp`, `/resend-otp`, all four `/forgot-password*` routes, `/api/profile/change-password*`, `/api/profile/delete*`, `/api/station-manager/access/setup`.
- Pass: an attacker rotating IPs is still limited per account, and an attacker rotating accounts is still limited per IP.

☐ RATE-06 Expensive non-auth endpoints are limited too

MEDIUM

- Endpoints: KYC upload, invoice PDF, statement export, station search, wallet order creation, booking creation, location ping.
- Pass: each has a limit proportional to its cost.

☐ RATE-07 429 responses are correct and non-informative

LOW

- Pass: status 429 with `Retry-After`; the body does not reveal whether the account exists or how many attempts remain.

☐ RATE-08 Buckets expire

LOW

- Pass: a TTL index on the bucket collection; verify the index exists in the deployed database.

☐ RATE-09 Limits are enforced across instances

HIGH

- Pass: Mongo-backed (shared), not in-memory. Confirm no in-process cache short-circuits the shared counter.

☐ RATE-10 Account lockout policy is deliberate

MEDIUM

- Method: decide and document whether repeated failures lock an account, and how a locked-out user recovers.
- Pass: whichever policy is chosen, it cannot be used by an attacker to lock out arbitrary users indefinitely.

16. Business-logic abuse

These are the bugs a scanner will never find. Walk each scenario end to end against a running instance.

☐ LOGIC-01 Book → charge → cancel to avoid payment

CRITICAL

- Attack: start a session, consume energy, then cancel the booking or delete the account before the bill settles.
- Pass: a bill is produced and remains owed; cancellation is refused once a session has started; account deletion does not erase an outstanding balance.

☐ LOGIC-02 Refund farming

HIGH

- Attack: repeatedly book and cancel within a refund window; cancel a booking whose promotional credit was already spent.
- Pass: refunds are idempotent, bounded, and cannot exceed what was actually paid.

☐ LOGIC-03 Free charging through state confusion

CRITICAL

- Attack: start a session, then have an admin/operator end it, force-free the connector, or change pricing mid-session; end the session twice; end it while the booking is being rescheduled.
- Pass: exactly one bill per session, computed from the snapshot, in every ordering.

☐ LOGIC-04 Withdraw credit that was never funded

CRITICAL

- Attack: obtain wallet credit from a refund or an admin adjustment, then withdraw it; withdraw during a pending top-up verification.
- Pass: withdrawable balance excludes unsettled and non-refundable credit; the check is atomic with the withdrawal.

☐ LOGIC-05 Connector denial of service

HIGH

- Attack: hold every connector at a station with back-to-back bookings that are never used.
- Pass: no-show expiry, per-customer concurrent booking caps, and (optionally) a penalty policy.

☐ LOGIC-06 Time manipulation

HIGH

- Attack: send timestamps in a distant timezone or with a manipulated offset to extend a booking window or start early.
- Pass: every decision uses the server clock (`AppClock`); client timestamps are advisory only.

☐ LOGIC-07 Race between reschedule approval and session start

HIGH

- Attack: request a reschedule and start a session at the same instant; have an admin approve while the session starts.
- Pass: the outcome is consistent under both orderings; no session runs against a booking window that no longer exists.

☐ LOGIC-08 Price change during an active booking

HIGH

- See BOOK-14 and STN-07. Confirm both directions (increase and decrease) behave per policy.

☐ LOGIC-09 KYC approval reversal

MEDIUM

- Attack: after approval and role elevation, reject the application, or delete it.
- Pass: transitions are one-way, or reversal explicitly demotes the role and revokes tokens.

☐ LOGIC-10 Self-approval through a second account

HIGH

- Attack: an admin who is also an applicant approves their own application from their admin account.
- Pass: separation of duties — the approver cannot be the applicant; enforced in code, not by policy alone.

□ **LOGIC-11** Negative or zero everywhere

HIGH

- Method: sweep every numeric input in the system with `0`, `-1`, and the maximum value — amounts, durations, kWh, radius, page size, thresholds, counts.
- Pass: each is bounded with an explicit constraint.

□ **LOGIC-12** Idempotency of every mutating endpoint

HIGH

- Method: replay every `POST` / `PUT` / `PATCH` twice, concurrently.
 - Pass: the second call is either rejected or a no-op; no duplicate bills, credits, notifications, sessions or applications.
-

17. Physical charger and protocol trust

☐ CHG-01 The charger integration is a stub and is labelled as one

CRITICAL

- Where: `BE/service/ChargerCommandService.java`.
- Pass: nothing in the product, marketing copy or documentation claims verified physical charging control today.

☐ CHG-02 Requirements before any real charger is connected

CRITICAL

- Authenticated OCPP (or the chosen protocol) with mutual authentication and per-charger identity.
- Start/stop commands acknowledged; unacknowledged commands never treated as executed.
- Charger, station and connector identities bound so one charger cannot report for another.
- Meter values signed or sourced from a trusted channel; client-reported energy never billed.
- Disconnect, reconnect and replay handling; physical state reconciled before a bill is issued.
- Firmware/update channel authenticated; no unauthenticated remote command surface.

☐ CHG-03 Billing tolerates charger dishonesty

HIGH

- Method: assume a compromised charger reports 10 000 kWh or 0 kWh.
 - Pass: bounds and anomaly detection on session energy, plus an operational review path — a single compromised charger must not be able to bill a customer arbitrarily.
-

18. Web frontend

Covers `plugin-frontend/`. Its invariant checker is `plugin-frontend/scripts/security-check.mjs` (`npm test`) — read it before touching auth storage, `ProtectedRoute`, `axios.js`, or the admin KYC view, and keep it passing.

☐ WEB-01 Bearer tokens live only in `sessionStorage`, via one module

HIGH

- Where: `FE/utils/authStorage.js`; `security-check.mjs` lines 30–31.
- Attack: grep for `localStorage.getItem('plugin_token')` and any direct token access outside `authStorage.js`.
- Pass: one storage boundary, per-tab `sessionStorage`, legacy `localStorage` entries deleted at startup.

☐ WEB-02 The XSS-to-token-theft risk is understood and contained

CRITICAL

- Status: documented residual risk in `plugin-frontend/SECURITY.md` — a JS-accessible token means any same-origin XSS is account takeover.
- Attack: grep for `dangerouslySetInnerHTML`, `eval`, `new Function`, `innerHTML`, `document.write`, and any `href` / `src` built from server data.
- Pass: none present. Target state is a short-lived access token plus a rotated `HttpOnly; Secure; SameSite` refresh cookie — record whether this is planned.

☐ WEB-03 User PII is not persisted in the browser

HIGH

- Where: `FE/context/AuthContext.jsx`; `security-check.mjs` line 29.
- Pass: no `plugin_user` object in `localStorage`; profile data is fetched, not cached across sessions.

☐ WEB-04 The bearer token is attached only to trusted origins

CRITICAL

- Where: `FE/api/axios.js` `assertTrustedRequestOrigin`; `security-check.mjs` line 28.
- Attack: set a `baseURL` override or a relative path that resolves to a foreign origin and confirm the token is withheld.
- Pass: the resolved URL and any `baseURL` override are both origin-checked before the header is added.

☐ WEB-05 `ProtectedRoute` uses exact role allow-lists

HIGH

- Where: `FE/components/ProtectedRoute/ProtectedRoute.jsx`; `security-check.mjs` lines 22–23.
- Pass: `allowedRoles.includes(user.role)`; no `role === 'ADMIN'` shortcut that implies operator access.

☐ WEB-06 A 403 does not destroy a valid session

MEDIUM

- Where: `security-check.mjs` line 32.
- Pass: only 401 clears the session; a 403 authorization denial does not log the user out (and does not mask an authorization bug as a login problem).

☐ WEB-07 The production build refuses insecure configuration

HIGH

- Where: `plugin-frontend/vite.config.js`; `security-check.mjs` lines 24–25.
- Attack: build with `VITE_API_BASE_URL` unset, `http://`, or containing credentials.
- Pass: the build fails.

☐ WEB-08 No secret is embedded in the web bundle

CRITICAL

- Attack: build and grep `dist/` for `key_secret`, `rzp_test`, `rzp_live`, `AIza`, `mongodb://`, `JWT`, SMTP credentials.
- Pass: only public identifiers (`VITE_GOOGLE_CLIENT_ID`, the Razorpay public key ID) appear.

☐ WEB-09 Response security headers are set at the edge

CRITICAL

- Status: open blocker in `docs/SECURITY-HARDENING.md` — the CloudFront response-headers policy.

- Required: `Content-Security-Policy` including `frame-ancestors 'none'`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`, `Strict-Transport-Security` (only once HTTPS is confirmed), and HTTP→HTTPS redirect.
- Pass: verified against the live URL with `curl -I`, not just asserted in a config file. A CSP `<meta>` tag alone is insufficient.

□ WEB-10 CORS is exact-origin and credential-free

HIGH

- Where: `BE/config/SecurityConfig.java` `corsConfigurationSource`, `BE-TEST/config/SecurityConfigCorsTest.java`.
- Attack: send `Origin: https://evil.com`, `Origin: null`, `https://myapp.com.evil.com`, and a subdomain of the real origin.
- Pass: rejected; wildcards impossible; `allowCredentials` stays `false`; production rejects loopback origins at startup.

□ WEB-11 Client-side routing does not leak admin data

MEDIUM

- Attack: log in as a customer and open every admin route directly.
- Pass: redirect plus a server-side 403 on the underlying API calls; no admin data is prefetched before the guard runs.

□ WEB-12 Third-party script surface is minimal

HIGH

- Attack: list every external script (Razorpay checkout, Google Sign-In, maps) loaded by the app.
- Pass: each is required, loaded from a pinned official origin over HTTPS, allow-listed in CSP, and ideally integrity-checked. Every third-party script on a page holding a JS-accessible token can steal it.

□ WEB-13 Dependencies are pinned and audited

MEDIUM

- Where: `security-check.mjs` line 36; `npm run audit:production`.
- Pass: all direct dependencies at exact versions; no unreviewed high or critical production advisory; a lockfile is committed and CI installs with `npm ci`.

□ WEB-14 Errors and source maps do not leak internals

MEDIUM

- Attack: check whether production serves `.map` files; trigger `AppErrorBoundary` and inspect what is rendered.
- Pass: no source maps in production, no stack traces or API internals shown to users.

□ WEB-15 Open redirect and URL-fragment handling are safe

HIGH

- Where: `FE/pages/StationManagerAccessSetup/StationManagerAccessSetup.jsx` reads the invitation token from the URL fragment.
- Attack: supply a `returnUrl / next` parameter pointing off-site; check whether the fragment token can be leaked via `Referer`, analytics, or a logged URL.
- Pass: redirects restricted to same-origin paths; the fragment is consumed and cleared from the URL immediately; no analytics or logging captures the full URL.

□ WEB-16 Clickjacking is prevented

MEDIUM

- Pass: `frame-ancestors 'none'` at the edge for the web app; the API already sends `X-Frame-Options: DENY` and a `frame-ancestors 'none'` CSP.

19. Mobile app

Covers `plugin_app/`. Its invariant checker is `plugin_app/scripts/security-check.cjs` (`npm test`, and `npm run security:release` for release mode).

☐ MOB-01 Credentials use platform secure storage

HIGH

- Where: `APP/api/client.js`; `security-check.cjs` lines 45–46.
- Pass: `expo-secure-store` for the bearer token; AsyncStorage is used only by the one-time migration helper.

☐ MOB-02 Release builds allow exactly one HTTPS API origin

CRITICAL

- Where: `security-check.cjs` lines 39–40, 71.
- Attack: build a release with an `http://` or credential-bearing URL, or with a stale remembered LAN URL.
- Pass: the release build fails; current configuration always takes precedence over a remembered development URL.

☐ MOB-03 Cleartext traffic and backup are disabled for release

HIGH

- Where: `security-check.cjs` lines 25–26, 32.
- Pass: `usesCleartextTraffic="false"` and `allowBackup="false"` in the main manifest; cleartext remains explicitly debug-only.

☐ MOB-04 Release signing is real and validated

CRITICAL

- Where: `security-check.cjs` lines 33–35, 75–76.
- Pass: only the debug build uses debug signing; the release task validates the certificate and rejects the debug certificate; keystore and passwords come from outside the repository.
- Also: `docs/SECURITY-HARDENING.md` records an Android signing migration blocker — existing debug-signed installs cannot be updated by a production certificate. Confirm the migration plan exists.

☐ MOB-05 No secret or key is hard-coded

CRITICAL

- Where: `security-check.cjs` lines 31, 48–50.
- Attack: grep the source and the built APK for `AIza`, `.apps.googleusercontent.com`, `rzp_`, and any bearer token.
- Pass: the Maps key is a build-time placeholder; the OAuth client ID comes from the environment; the Maps key is restricted by package name, signing certificate, API and quota.

☐ MOB-06 The development test-confirm path is compiled out

CRITICAL

- Where: `security-check.cjs` lines 44, 47.
- Pass: no `/wallet/mandate/test-confirm` string in the release client; the Razorpay test confirmation is `__DEV__`-only.

☐ MOB-07 Mutating requests are never auto-retried

HIGH

- Where: `security-check.cjs` line 38.
- Attack: cause a timeout during top-up verification or session end and check for a duplicate effect.
- Pass: only `GET` / `HEAD` / `OPTIONS` fall back or retry.

☐ MOB-08 A 403 does not clear mobile auth

MEDIUM

- Where: `security-check.cjs` line 53. Same rationale as WEB-06.

☐ MOB-09 Deep links and exported components are minimal

HIGH

- Where: `security-check.cjs` line 52.
- Attack: enumerate exported activities/receivers and any custom scheme; attempt to launch a screen with attacker-controlled parameters.

- Pass: no unused custom-scheme intent filters; no exported component performs a privileged action from an intent extra.

☐ **MOB-10 The offline session cache is display-only and cleared on logout**

MEDIUM

- See SESS-10. Confirm `plugin_active_session_snapshot` holds no token and no other user's data.

☐ **MOB-11 Nothing sensitive is written to device logs**

HIGH

- Attack: run a release build with `adb logcat` while logging in, topping up, and charging.
- Pass: no token, OTP, payment identifier, coordinate or PII in the log.

☐ **MOB-12 Screenshots and the app switcher do not expose data**

LOW

- Pass: consider `FLAG_SECURE` on wallet, payment and KYC screens; at minimum, decide deliberately.

☐ **MOB-13 Location permission use is minimal and explained**

MEDIUM

- Where: `APP/screens/StationNavigationScreen.js`, booking location pings.
- Pass: foreground-only where possible, requested at the point of use, stopped when the booking ends, and disclosed in the privacy policy.

☐ **MOB-14 Certificate and transport posture**

MEDIUM

- Pass: system trust store with HTTPS enforced; consider pinning for the API host, accepting the operational cost. No custom `TrustManager` that accepts all certificates — grep for one.

☐ **MOB-15 Dependencies are pinned and Expo-compatible**

MEDIUM

- Where: `security-check.cjs` lines 36–37, 56, 60.
- Pass: exact versions, a checksum-verified Gradle wrapper, and versions inside Expo's compatible ranges.

☐ **MOB-16 The APK resists trivial tampering expectations**

LOW

- Pass: no security decision depends on client integrity. Root/emulator detection is optional; server-side enforcement is mandatory.
-

20. Data layer, MongoDB and privacy

☐ DATA-01 The database user is least-privilege

CRITICAL

- Pass: the application's Atlas user has read/write on exactly one database, no admin or cluster role, and cannot drop collections or create users.

☐ DATA-02 Network access to Atlas is restricted

CRITICAL

- Attack: attempt a connection from an unlisted address; check the IP access list for `0.0.0.0/0`.
- Pass: allow-list or private endpoint only, TLS required, SCRAM or certificate authentication.

☐ DATA-03 Unique indexes actually exist in the deployed database

HIGH

- Note: `spring.data.mongodb.auto-index-creation: false` — annotations alone create nothing.
- Method: run `db.<collection>.getIndexes()` for `users.email`, `users.id`, the Razorpay payment/order identifiers, the KYC reference ID, invoice numbers, and the application-file compound key.
- Pass: each expected unique index is present. `docs/SECURITY-HARDENING.md` flags duplicate-data reconciliation as a prerequisite — reconcile on a staging clone, take a tested backup, then enable.

☐ DATA-04 TTL indexes exist for ephemeral collections

MEDIUM

- Collections: `PendingRegistration`, `PasswordResetOtp`, `SecurityRateLimitBucket`, `StationManagerAccessInvitation`, `WalletTopUpAttempt`.
- Pass: TTL indexes present so secrets and stale attempts do not accumulate indefinitely.

☐ DATA-05 Multi-document invariants run in transactions on a replica set

HIGH

- Attack: run the wallet and session flows against a standalone MongoDB — transactions silently degrade.
- Pass: production and the concurrency test environment both use a replica set; the application fails loudly rather than proceeding non-transactionally.

☐ DATA-06 Sensitive fields are protected at rest

HIGH

- Pass: encryption at rest enabled; consider field-level encryption or a KMS for identity documents, phone numbers and vehicle registrations. Record the decision.

☐ DATA-07 Backups exist, are encrypted, and restore has been tested

HIGH

- Pass: automated backups, a documented retention period, restricted restore access, and at least one rehearsed restore.

☐ DATA-08 No production data in development environments

HIGH

- Pass: seeds and fixtures are synthetic; no real customer or KYC data is copied to a laptop or a test cluster.

☐ DATA-09 Data subject rights are implementable

MEDIUM

- Pass: export and deletion procedures exist for a customer's personal data across users, vehicles, bookings, sessions, bills, wallet, notifications and KYC — with the financial-retention exception documented.

☐ DATA-10 PII is not written to application logs

HIGH

- Attack: run the main flows at `INFO` and grep the log for emails, phones, coordinates, registrations and document names.
- Pass: identifiers only; a documented masking policy.

☐ **DATA-11** Response DTOs never expose internal fields

HIGH

- Method: diff every entity against its response DTO — check for `password`, `tokenVersion`, `googleSubject`, `mongoId`, internal notes, and raw file paths.
- Pass: entities are never serialised directly to a client.

☐ **DATA-12** Legacy numeric IDs do not leak business volume

LOW

- Note: `database_sequences` yields sequential IDs that reveal customer and booking counts.
 - Pass: acceptable if deliberate; otherwise expose an opaque public identifier.
-

21. Configuration, infrastructure and delivery

☐ CFG-01 The application fails closed without its secrets

CRITICAL

- Pass: startup fails without `JWT_SECRET`, `OTP_PEPPER`, `RATE_LIMIT_PEPPER` (each ≥ 32 random bytes, independent values). Verify by starting with each unset.

☐ CFG-02 `APP_PRODUCTION=true` in production and it changes behaviour

CRITICAL

- Pass: production rejects loopback CORS origins, rejects Razorpay test keys, and blocks the mandate test-confirm path. Verify on the deployed instance, not locally.

☐ CFG-03 No secret is committed, defaulted or logged

CRITICAL

- Attack: `git log -p` and `git grep` across full history for `JWT_SECRET`, `OTP_PEPPER`, `RATE_LIMIT_PEPPER`, `mongodb+srv://`, `rzp_`, `AIza`, `MAIL_PASSWORD`, `.env`, `.keystore`, `application-local.yml`.
- Pass: nothing found in history. **Any credential ever committed, pasted, logged, emailed or shared during development must be rotated** — Mongo, SMTP, Razorpay, Google Maps, JWT/OTP/rate-limit keys, cloud and deployment keys. Note the defaulted SMTP username in `BE-CFG` and confirm the real mailbox credential has been rotated.

☐ CFG-04 `CORS_ALLOWED_ORIGINS` lists exactly the real origins

HIGH

- Pass: no wildcard, no path, no localhost in production, HTTPS only.

☐ CFG-05 `TRUSTED_PROXY_CIDRS` is narrow

CRITICAL

- Pass: only the immediate load balancer's addresses; never a broad public range added to "make forwarded IPs work". See RATE-02.

☐ CFG-06 TLS is correct end to end

CRITICAL

- Attack: `curl -I http://<api-host>` and `https://<api-host>`; test the certificate chain; attempt TLS 1.0/1.1.
- Pass: HTTP redirects to HTTPS, a valid chain, modern protocol versions only, HSTS enabled once HTTPS is confirmed, and the backend not directly reachable on a public port without TLS.

☐ CFG-07 Backend security headers survive the edge

HIGH

- Pass: the headers set in `SecurityConfig` (CSP, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, HSTS) arrive intact at the client and are not stripped or duplicated by the proxy.

☐ CFG-08 The management/actuator surface is closed

HIGH

- Attack: request `/actuator`, `/actuator/env`, `/actuator/health`, `/actuator/heapdump`, `/error`, `/v3/api-docs`, `/swagger-ui`.
- Pass: absent or authenticated. `/actuator/env` and `/heapdump` expose secrets outright.

☐ CFG-09 The API server is not directly exposed

HIGH

- Note: `server.address` defaults to `0.0.0.0:8091`.
- Pass: a security group or firewall permits only the load balancer; the instance port is not reachable from the internet.

☐ CFG-10 Uploads are stored outside the deployment artifact

CRITICAL

- See KYC-12. Confirm the path used by the deployed unit and that a redeploy neither exposes nor destroys files.

❑ **CFG-11** The container and host are hardened

MEDIUM

- Where: `plugin-backend/Dockerfile`.
- Pass: a pinned non-`latest` base image, a non-root user, no build secrets in layers, a current JRE, and a documented patching cadence.

❑ **CFG-12** Deployment integrity is verified

HIGH

- Where: `.github/workflows/deploy-backend.yml`, `deploy-frontend.yml`.
- Pass: SSH host keys verified, artifact checksums verified, actions pinned to commit SHAs, build jobs secretless, frontend AWS access via OIDC, and a health-check failure triggering rollback.

❑ **CFG-13** CI secrets are scoped and unprintable

HIGH

- Attack: review workflow logs for echoed secrets; check whether a pull request from a fork can reach production secrets.
- Pass: environment-scoped secrets, no `pull_request_target` misuse, no secret interpolation into a shell string that could be logged.

❑ **CFG-14** The security CI gate actually blocks

HIGH

- Where: `.github/workflows/security-ci.yml`.
- Pass: it runs `mvn clean verify`, both `security-check` scripts and the dependency audits, and a failure blocks the merge — it is not `continue-on-error`.

❑ **CFG-15** Dependencies are current and monitored

MEDIUM

- Pass: backend and both client dependency audits are clean of unreviewed high/critical findings; automated update alerts are enabled; a Java/Spring Boot patch cadence is agreed.

❑ **CFG-16** Email transport is authenticated

MEDIUM

- Pass: SMTP over STARTTLS with `required: true` (as configured), credentials from a secret store, SPF/DKIM/DMARC published for the sending domain so OTP mail is not trivially spoofable.

❑ **CFG-17** `WEBSITE_URL` and `APP_BASE_URL` are exact production HTTPS values

HIGH

- Note: the defaults are `www.plugin.com` and `http://localhost:8091`.
- Pass: both set explicitly in production; invitation and confirmation links cannot be built against a wrong or attacker-influenced host.

❑ **CFG-18** Multi-instance behaviour is understood

HIGH

- Method: list every `@Scheduled` job — booking expiry, session auto-complete, notifications, wallet monitor.
 - Pass: each has a distributed lease, or the deployment is pinned to a single instance and that constraint is documented and enforced.
-

22. Logging, monitoring and incident response

☐ OBS-01 Security events are logged

HIGH

- Events: authentication failure, logout/429, authorization denial (403), token rejection reason class, KYC document view, admin action, payment verification failure, rate-limiter storage failure.

☐ OBS-02 Logs are centralised, tamper-evident and retained

MEDIUM

- Pass: shipped off-host, access-restricted, retained long enough to investigate an incident.

☐ OBS-03 Alerting exists for attack signatures

MEDIUM

- Signals: a spike in 401/403/429, repeated failed payment verifications, one account touching many resources, rate-limit storage unavailability, an unusual admin action volume.

☐ OBS-04 Correlation IDs tie a client error to a server log

LOW

- Pass: a request ID is returned in error responses and recorded in the log, so support does not need verbose client errors.

☐ OBS-05 An incident response plan exists

HIGH

- Contents: who is on call, how to revoke all tokens (rotate `JWT_SECRET` / bump `tokenVersion`), how to disable a compromised operator, how to freeze payouts, breach notification obligations and timelines.

☐ OBS-06 Key rotation is scheduled and rehearsed

HIGH

- Pass: documented owners and dates for JWT/OTP/rate-limit secrets, Mongo, SMTP, Razorpay, Maps and deployment keys; rotation tested at least once.

☐ OBS-07 A vulnerability disclosure channel exists

LOW

- Pass: a published security contact and a stated response commitment.

☐ OBS-08 Penetration test before public launch

HIGH

- Pass: an independent test covering authentication, authorization/IDOR, payments and KYC, with findings tracked to closure and a retest.
-

23. Verification commands

Run these after every batch of fixes. All three suites must pass.

```
cd plugin-backend
mvn clean verify

cd plugin-frontend
npm ci
npm test
npm run audit:production
$env:VITE_API_BASE_URL="https://api.example.com/api"
$env:VITE_GOOGLE_CLIENT_ID="your_web_client_id.apps.googleusercontent.com"
npm run build

cd plugin_app
npm ci
npm test
$env:PLUGIN_BUILD_PROFILE="production"
npm run security:release
```

Tests that cannot be expressed statically, and must be run manually against a replica-set MongoDB:

- Two concurrent bookings for the same connector and overlapping window.
 - Concurrent wallet top-up verification and bill payment on one wallet.
 - Duplicate Razorpay payment identifier submitted twice.
 - Session start and end ownership across two accounts.
 - Cross-account KYC file and status access.
 - Malicious upload samples (polyglot image, active PDF, SVG with script, renamed executable).
 - Expired and reused station-manager invitation.
 - OTP guessing beyond the attempt limit, and resend flooding.
 - Rate-limit storage failure while under load (must return 503).
 - Two backend instances running the schedulers against one database.
 - A failed deployment rolling back cleanly.
-

24. Reporting format

Return findings in this shape, most severe first. One row per checklist item that is not `PASS`.

```
ID:      PAY-03
Status:   FAIL
Severity: CRITICAL
Evidence: plugin-backend/src/main/java/com/plugin/service/WalletService.java:142
Exploit:  Replaying razorpay_payment_id X credits the wallet a second time because
          the uniqueness constraint exists only as an annotation and the index was
          never created (auto-index-creation is false).
Fix:      Create the unique index on walletTopUpAttempts.razorpayPaymentId after
          reconciling duplicates; make the credit path a conditional atomic update
          keyed on that identifier.
Test:     BE-TEST/service/WalletServiceSecurityTest#replayedPaymentCreditsOnce
Residual: None once the index is deployed and verified with getIndexes().
```

End the report with:

1. **Counts** — findings by severity, and the number of items in each of PASS / FAIL / N/A / NEEDS-EVIDENCE.
2. **Launch blockers** — every CRITICAL and HIGH still open.
3. **External blockers** — items that cannot be closed in code (OCPP, webhooks, TLS/edge headers, AV scanning, MFA, secret rotation, Android signing migration, Mongo index rollout, privacy lifecycle). Cross-check against `docs/SECURITY-HARDENING.md` and update that document with the current truth.
4. **A single sentence** stating whether the system is safe for public launch, with the specific reasons if not.

This checklist is a testing instrument, not a certificate. Every item passing means the listed attack paths were closed and proven closed — it does not prove the absence of all vulnerabilities. Re-run it after any change to authentication, authorization, payments, KYC handling, or the booking and session state machines.